```
===== backend/orchestrator/app/__init__.py =====
# Orchestrator package

===== backend/orchestrator/app/__main__.py =====
from app.main import run

if __name__ == "__main__":
    run()

===== backend/orchestrator/app/audio/__init__.py =====
# audio helpers

===== backend/orchestrator/app/audio/pcm.py =====
from __future__ import annotations

import time
from dataclasses import dataclass, field


def now_ms() -> int:
    return int(time.perf_counter() * 1000)


@dataclass
class LatencyProbe:
    """Per-utterance timestamps, milliseconds on the orchestrator clock."""

    utterance_id: str
    t0: int = field(default_factory=now_ms)
    vad_end: int | None = None
    stt_first_partial: int | None = None
    stt_final: int | None = None
    mt_done: int | None = None
    tts_first_byte: int | None = None

    def mark(self, stage: str) -> None:
        setattr(self, stage, now_ms())

    def stages_ms(self) -> dict[str, int]:
        out: dict[str, int] = {}
        for name in (
            "vad_end",
            "stt_first_partial",
            "stt_final",
            "mt_done",
            "tts_first_byte",
        ):
            value = getattr(self, name)
            if value is not None:
                out[name] = value - self.t0
        if self.tts_first_byte is not None:
            out["e2e_server_ms"] = self.tts_first_byte - self.t0
        return out

===== backend/orchestrator/app/audio/vad.py =====
```

```
"""Voice activity detection.

Default: Silero VAD (ONNX) \u2014 better Indic speech / noise rejection than
WebRTC VAD, native 16 kHz, 32 ms windows. Fallback: webrtcvad.

Silero is the endpointing clock. Sub-1 s latency lives or dies here:
cut after ~180 ms of silence, not 400\u2013800 ms.
"""

from __future__ import annotations

from pathlib import Path

import numpy as np
import structlog

from app.config import Settings

log = structlog.get_logger("vad")

SILERO_WINDOW = 512  # 32 ms @ 16 kHz (Silero v4/v5)
SILERO_ONNX_URL = (
    "https://github.com/snakers4/silero-vad/raw/master/src/silero_vad/data/silero_vad.onnx"
)



class EndpointingVad:
    def __init__(self, settings: Settings) -> None:
        self._settings = settings
        self._frame_ms = settings.frame_ms
        self._silence_needed = max(1, settings.vad_silence_ms // settings.frame_ms)
        self._min_speech = max(1, settings.vad_min_speech_ms // settings.frame_ms)
        self._max_frames = max(1, settings.vad_max_utterance_ms // settings.frame_ms)
        self._threshold = settings.vad_threshold
        self._backend = settings.vad_backend
        self._silero: _SileroOnnx | None = None
        self._webrtc = None
        if settings.vad_backend == "silero":
            try:
                self._silero = _SileroOnnx(settings)
            except Exception as exc:  # noqa: BLE001
                log.warning("silero.unavailable", error=str(exc), fallback="webrtc")
                self._backend = "webrtc"
        if self._backend == "webrtc":
            import webrtcvad

            self._webrtc = webrtcvad.Vad(settings.vad_aggressiveness)
        self.reset()

    def reset(self) -> None:
        self._in_speech = False
        self._silence_run = 0
        self._speech_frames = 0
        self._buffer = bytearray()
        self._pcm_window = bytearray()
```

```
        if self._silero:
            self._silero.reset()

    def push(self, pcm: bytes) -> tuple[bool, bool, bytes | None]:
        """Returns (speech_started, utterance_ended, utterance_pcm_or_None)."""
        if len(pcm) < 2:
            return False, False, None

        voiced = self._is_voiced(pcm)
        self._buffer.extend(pcm)
        started = False

        if voiced:
            if not self._in_speech:
                started = True
            self._in_speech = True
            self._silence_run = 0
            self._speech_frames += 1
        elif self._in_speech:
            self._silence_run += 1

        ended = False
        if (
            self._in_speech
            and self._speech_frames >= self._min_speech
            and self._silence_run >= self._silence_needed
        ):
            ended = True
        if self._speech_frames >= self._max_frames:
            ended = True

        if not ended:
            return started, False, None

        utterance = bytes(self._buffer)
        self.reset()
        return started, True, utterance

    def _is_voiced(self, pcm: bytes) -> bool:
        if self._silero is not None:
            self._pcm_window.extend(pcm)
            need = SILERO_WINDOW * 2
            if len(self._pcm_window) < need:
                return self._in_speech
            window = bytes(self._pcm_window[-need:])
            self._pcm_window = bytearray(window)
            return self._silero.prob(window) >= self._threshold
        assert self._webrtc is not None
        try:
            return bool(self._webrtc.is_speech(pcm, self._settings.sample_rate_hz))
        except Exception:
            return False


class _SileroOnnx:
```

```
    def __init__(self, settings: Settings) -> None:
        import onnxruntime as ort

        path = Path(settings.silero_onnx_path)
        if not path.exists():
            path.parent.mkdir(parents=True, exist_ok=True)
            _download(SILERO_ONNX_URL, path)
        opts = ort.SessionOptions()
        opts.inter_op_num_threads = 1
        opts.intra_op_num_threads = 1
        self._session = ort.InferenceSession(str(path), opts, providers=["CPUExecutionProvider"])
        self._inputs = [i.name for i in self._session.get_inputs()]
        self._sr = np.array(settings.sample_rate_hz, dtype=np.int64)
        self.reset()

    def reset(self) -> None:
        # v4: h,c (2,1,64)  |  v5: state (2,1,128)
        if "state" in self._inputs:
            self._state = np.zeros((2, 1, 128), dtype=np.float32)
            self._h = self._c = None
        else:
            self._h = np.zeros((2, 1, 64), dtype=np.float32)
            self._c = np.zeros((2, 1, 64), dtype=np.float32)
            self._state = None

    def prob(self, pcm16: bytes) -> float:
        audio = np.frombuffer(pcm16, dtype=np.int16).astype(np.float32) / 32768.0
        if audio.size < SILERO_WINDOW:
            audio = np.pad(audio, (0, SILERO_WINDOW - audio.size))
        audio = audio[:SILERO_WINDOW].reshape(1, -1)
        if self._state is not None:
            out, self._state = self._session.run(
                None, {"input": audio, "state": self._state, "sr": self._sr}
            )[:2]
        else:
            out, self._h, self._c = self._session.run(
                None, {"input": audio, "h": self._h, "c": self._c, "sr": self._sr}
            )[:3]
        return float(out.squeeze())


def _download(url: str, dest: Path) -> None:
    import urllib.request

    log.info("silero.download", url=url, dest=str(dest))
    urllib.request.urlretrieve(url, dest)

===== backend/orchestrator/app/config.py =====
from __future__ import annotations

from functools import lru_cache
from typing import Literal

from pydantic import Field
from pydantic_settings import BaseSettings, SettingsConfigDict
```

```
ProviderName = Literal["google", "google_v2", "azure", "sarvam", "openai", "anthropic", "mock"]
VadBackend = Literal["silero", "webrtc"]


class Settings(BaseSettings):
    model_config = SettingsConfigDict(
        env_file=".env",
        extra="ignore",
        populate_by_name=True,
    )

    host: str = Field(default="0.0.0.0", validation_alias="ORCHESTRATOR_HOST")
    port: int = Field(default=8090, validation_alias="ORCHESTRATOR_PORT")

    # Convenience preset. Individual *_provider fields win when set to a
    # non-empty override via STT_PROVIDER / TRANSLATOR_PROVIDER / TTS_PROVIDER.
    pipeline_provider: Literal["google", "azure", "sarvam", "mock"] = "mock"
    stt_provider: str = ""
    translator_provider: str = ""
    tts_provider: str = ""

    sample_rate_hz: int = 16000
    frame_ms: int = 20
    target_e2e_ms: int = 1000

    google_project_id: str = ""
    google_stt_model: str = "latest_short"
    google_stt_location: str = "global"

    azure_speech_key: str = ""
    azure_speech_region: str = "centralindia"
    azure_translator_key: str = ""
    azure_translator_region: str = "centralindia"
    azure_translator_endpoint: str = "https://api.cognitive.microsofttranslator.com"

    sarvam_api_key: str = ""
    sarvam_stt_model: str = "saaras:v3-realtime"
    sarvam_tts_model: str = "bulbul:v3"
    sarvam_tts_speaker: str = "anushka"
    sarvam_base_url: str = "https://api.sarvam.ai"

    openai_api_key: str = ""
    openai_model: str = "gpt-4o-mini"
    openai_base_url: str = "https://api.openai.com/v1"

    anthropic_api_key: str = ""
    anthropic_model: str = "claude-haiku-4-5"

    vad_backend: VadBackend = "silero"
    vad_aggressiveness: int = 2
    vad_silence_ms: int = 180
    vad_min_speech_ms: int = 120
    vad_max_utterance_ms: int = 1800
```

```
    vad_threshold: float = 0.45
    silero_onnx_path: str = "models/silero_vad.onnx"
    partial_translate: bool = True
    partial_min_chars: int = 4
    half_duplex: bool = True
    barge_in: bool = True
    # Forward the speaker's original voice to the listener while no translation is playing.
    # Off: the listener hears only the translation, as the architecture intends.
    relay_original_voice: bool = False
    # Ignore a speaker's mic while their translated voice plays on the other phone. Only for
    # two phones in the same room, where each mic hears the other's speaker.
    crosstalk_guard: bool = False

    @property
    def frame_bytes(self) -> int:
        return int(self.sample_rate_hz * (self.frame_ms / 1000.0) * 2)

    @property
    def samples_per_frame(self) -> int:
        return int(self.sample_rate_hz * (self.frame_ms / 1000.0))

    def resolved_stt(self) -> str:
        return _choose(self.stt_provider, _preset_stt(self._preset()), {"google", "google_v2",
    "azure", "sarvam", "mock"})

    def resolved_translator(self) -> str:
        return _choose(
            self.translator_provider,
            _preset_mt(self._preset()),
            {"google", "azure", "sarvam", "openai", "anthropic", "mock"},
        )

    def resolved_tts(self) -> str:
        return _choose(self.tts_provider, _preset_tts(self._preset()), {"google", "azure", "sarvam",
     "mock"})

    def _preset(self) -> str:
        name = self.pipeline_provider.split("#", 1)[0].strip().lower()
        return name if name in {"google", "azure", "sarvam", "mock"} else "mock"


def _choose(override: str, preset_value: str, allowed: set[str]) -> str:
    name = override.split("#", 1)[0].strip().lower()
    if name in allowed:
        return name
    return preset_value


def _preset_stt(preset: str) -> str:
    return {"google": "google_v2", "azure": "azure", "sarvam": "sarvam", "mock": "mock"}[preset]


def _preset_mt(preset: str) -> str:
    return {"google": "google", "azure": "azure", "sarvam": "sarvam", "mock": "mock"}[preset]
```

```
def _preset_tts(preset: str) -> str:
    return {"google": "google", "azure": "azure", "sarvam": "sarvam", "mock": "mock"}[preset]


@lru_cache(maxsize=1)
def get_settings() -> Settings:
    return Settings()

===== backend/orchestrator/app/hub.py =====
from __future__ import annotations

import asyncio
import time
from dataclasses import dataclass, field

from fastapi import WebSocket

from app.protocol import dumps

# Covers the client playback buffer plus a 200 ms tail after the speaker stops.
ECHO_TAIL_S = 0.35


@dataclass
class PeerSocket:
    peer_id: str
    ws: WebSocket

    async def send(self, payload: bytes | str) -> None:
        try:
            if isinstance(payload, (bytes, bytearray)) and payload[:1] in (b"{", b"["):
                await self.ws.send_text(payload.decode())
            elif isinstance(payload, (bytes, bytearray)):
                await self.ws.send_bytes(payload)
            else:
                await self.ws.send_text(payload)
        except Exception:
            return


@dataclass
class CallRoom:
    call_id: str
    peers: dict[str, PeerSocket] = field(default_factory=dict)
    lock: asyncio.Lock = field(default_factory=asyncio.Lock)
    # peer_id -> monotonic time until which that peer's mic is treated as echo.
    echo_guard_until: dict[str, float] = field(default_factory=dict)
    # peer_id -> monotonic time when everything already queued for that peer finishes playing.
    playback_end: dict[str, float] = field(default_factory=dict)

    async def add(self, peer: PeerSocket) -> None:
        async with self.lock:
            previous = self.peers.get(peer.peer_id)
            self.peers[peer.peer_id] = peer
```

```
        if previous and previous.ws is not peer.ws:
            try:
                await previous.ws.close(code=4001)
            except Exception:
                return

    async def remove(self, peer_id: str, ws: WebSocket | None = None) -> None:
        async with self.lock:
            current = self.peers.get(peer_id)
            if current is None:
                return
            if ws is not None and current.ws is not ws:
                return
            self.peers.pop(peer_id, None)
            self.echo_guard_until.pop(peer_id, None)
            self.playback_end.pop(peer_id, None)

    def mark_receivers_playing(self, speaker_id: str, pcm_bytes: int, sample_rate_hz: int) -> None:
        """Ignore the listeners' mics while this speaker's audio is still coming out of their
    speaker."""
        seconds = pcm_bytes / float(sample_rate_hz * 2) if pcm_bytes else 0.02
        now = time.monotonic()
        for peer_id in list(self.peers):
            if peer_id == speaker_id:
                continue
            # Speech is made faster than it plays, so chunks queue up on the phone. The mic stays
            # guarded until the whole queue has played, not just this chunk.
            end = max(now, self.playback_end.get(peer_id, 0.0)) + seconds
            self.playback_end[peer_id] = end
            self.echo_guard_until[peer_id] = end + ECHO_TAIL_S

    def clear_playback(self, speaker_id: str) -> None:
        """The speaker's voice was cancelled, so listeners' phones dropped what was queued."""
        now = time.monotonic()
        for peer_id in list(self.peers):
            if peer_id != speaker_id:
                self.playback_end[peer_id] = now
                self.echo_guard_until[peer_id] = now + ECHO_TAIL_S

    def audible_elsewhere(self, speaker_id: str) -> bool:
        """True while this speaker's audio is still playing on another phone."""
        now = time.monotonic()
        return any(
            now < end + ECHO_TAIL_S
            for peer_id, end in self.playback_end.items()
            if peer_id != speaker_id
        )

    def uplink_is_echo(self, peer_id: str) -> bool:
        return time.monotonic() < self.echo_guard_until.get(peer_id, 0.0)

    async def send_to(self, peer_id: str, payload: bytes | str) -> None:
        peer = self.peers.get(peer_id)
        if peer:
            await peer.send(payload)
```

```
    async def send_others(self, sender_id: str, payload: bytes | str) -> None:
        for pid, peer in list(self.peers.items()):
            if pid != sender_id:
                await peer.send(payload)

    async def broadcast_control(self, payload: dict) -> None:
        raw = dumps(payload)
        for peer in list(self.peers.values()):
            await peer.send(raw)


class Hub:
    def __init__(self) -> None:
        self._rooms: dict[str, CallRoom] = {}
        self._lock = asyncio.Lock()

    async def room(self, call_id: str) -> CallRoom:
        async with self._lock:
            if call_id not in self._rooms:
                self._rooms[call_id] = CallRoom(call_id=call_id)
            return self._rooms[call_id]

    async def drop_if_empty(self, call_id: str) -> None:
        async with self._lock:
            room = self._rooms.get(call_id)
            if room and not room.peers:
                self._rooms.pop(call_id, None)


hub = Hub()
```

```
===== backend/orchestrator/app/languages.py =====
"""One map for speech recognition, translation, and the voice that speaks the result.

Hindi and the other Indic languages name a native script. The translator must
ask for that script. A Hindi voice reading Latin letters sounds like Indian English.
"""

from __future__ import annotations

from dataclasses import dataclass


@dataclass(frozen=True, slots=True)
class LanguageProfile:
    name: str
    stt_code: str
    translator_code: str
    tts_voice: str
    script: str | None = None
    to_script: str | None = None


LANGUAGES: tuple[LanguageProfile, ...] = (
```

```
    LanguageProfile("Hindi", "hi-IN", "hi", "hi-IN-SwaraNeural", "Devanagari", "Deva"),
    LanguageProfile("Tamil", "ta-IN", "ta", "ta-IN-PallaviNeural", "Tamil", "Taml"),
    LanguageProfile("Telugu", "te-IN", "te", "te-IN-ShrutiNeural", "Telugu", "Telu"),
    LanguageProfile("Malayalam", "ml-IN", "ml", "ml-IN-SobhanaNeural", "Malayalam", "Mlym"),
    LanguageProfile("Gujarati", "gu-IN", "gu", "gu-IN-DhwaniNeural", "Gujarati", "Gujr"),
    LanguageProfile("Marathi", "mr-IN", "mr", "mr-IN-AarohiNeural", "Devanagari", "Deva"),
    LanguageProfile("Bengali", "bn-IN", "bn", "bn-IN-TanishaaNeural", "Bengali", "Beng"),
    LanguageProfile("English (India)", "en-IN", "en", "en-IN-NeerjaNeural"),
    LanguageProfile("German", "de-DE", "de", "de-DE-KatjaNeural"),
    LanguageProfile("Chinese Mandarin", "zh-CN", "zh-Hans", "zh-CN-XiaoxiaoNeural"),
    LanguageProfile("French", "fr-FR", "fr", "fr-FR-DeniseNeural"),
    LanguageProfile("Spanish", "es-ES", "es", "es-ES-ElviraNeural"),
    LanguageProfile("Dutch", "nl-NL", "nl", "nl-NL-ColetteNeural"),
    LanguageProfile("Portuguese (Portugal)", "pt-PT", "pt-pt", "pt-PT-RaquelNeural"),
    LanguageProfile("Portuguese (Brazil)", "pt-BR", "pt", "pt-BR-FranciscaNeural"),
)

_BY_CODE: dict[str, LanguageProfile] = {}
for _item in LANGUAGES:
    _BY_CODE[_item.stt_code.lower()] = _item
    _BY_CODE.setdefault(_item.stt_code.split("-")[0].lower(), _item)


def lookup(code: str | None) -> LanguageProfile | None:
    if not code:
        return None
    key = code.strip().lower()
    found = _BY_CODE.get(key)
    if found:
        return found
    return _BY_CODE.get(key.split("-")[0])


def voice_for(code: str | None) -> str:
    """Native TTS voice for this language. Never an English voice for Hindi."""
    profile = lookup(code)
    if profile:
        return profile.tts_voice
    fallback = (code or "en-IN").strip()
    return f"{fallback}-Neural"


def translator_pair(src: str, dst: str) -> tuple[str, str, str | None]:
    source = None if is_auto(src) else lookup(src)
    target = lookup(dst)
    if source:
        from_code = source.translator_code
    elif is_auto(src):
        from_code = ""
    else:
        from_code = src.split("-")[0]
    to_code = target.translator_code if target else dst.split("-")[0]
    to_script = target.to_script if target else None
    return from_code, to_code, to_script
```